

The Range Shrinks, the Threat Remains:

Re-evaluating LLM Package Hallucinations on the 2026 Frontier-Model Cohort

Aleksandr Churilov

Independent Researcher

churik0509@gmail.com

Abstract

Spracklen et al. (USENIX Security '25) showed that code-generating large language models hallucinate package names that do not exist on PyPI or npm at rates ranging from 5.2% on commercial models to 21.7% on open-source models, creating an attack surface for slopsquatting-the registration of malicious packages under hallucinated names. We replicate their methodology on five frontier code-capable LLMs released between October 2025 and March 2026: Claude Sonnet 4.6, Claude Haiku 4.5, GPT-5.4-mini, Gemini 2.5 Pro, and DeepSeek V3.2. Across 199,845 paired Python and JavaScript prompts validated against PyPI and npm master lists, we measure overall hallucination rates between 4.62% (Claude Haiku 4.5) and 6.10% (GPT-5.4-mini)-an order-of-magnitude compression of the inter-model spread observed by Spracklen, but not a retirement of the threat. Beyond replication, we identify a set of 127 package names (109 on PyPI, 18 on npm) that all five evaluated models invent identically; following coordinated disclosure with PyPI Security and Socket.dev, 53 of these (41 on PyPI, 12 on npm) remain registrable by an attacker after each registry's existing defenses, constituting a model-agnostic supply-chain attack surface that no single-model study can reveal. We further document a Python-over-JavaScript hallucination asymmetry that inverts Spracklen's 2024 finding, identify a Haiku-below-Sonnet inversion within the Anthropic family, and observe a Jaccard-similarity peak between DeepSeek V3.2 and GPT-5.4-mini ($J = 0.343$) suggestive of shared training-data origins.

1 Introduction

Modern software development depends on centralized package registries: PyPI for Python and npm for JavaScript host millions of packages installable with a single command. The emergence of code-generating large language models has injected a new actor into this supply chain. When a developer asks an LLM for code, the model frequently emits `pip install` or `npm install` directives, or import statements that name particular packages. If the developer trusts the suggestion and installs the named package, an attacker who has registered a malicious package under that name has succeeded-without ever interacting with the developer or the LLM provider.

Spracklen et al. [1] characterized this risk on the September-2024 model cohort, generating 576,000 code samples across 16 models and finding that 21.7% of open-source-model generations and 5.2% of commercial-model generations referenced non-existent packages. They identified 205,474 unique hallucinated package names and termed the resulting attack class slopsquatting: the registration of malicious packages under names that LLMs have already, by their hallucination behavior, primed downstream developers to install.

Two model generations later, the frontier has moved. Anthropic, OpenAI, Google, and DeepSeek have all released

code-capable models in the eighteen months following Spracklen's study, with substantially expanded training data, broader safety post-training, and-in the cases of Claude Haiku 4.5, GPT-5.4-mini, and DeepSeek V3.2-newly available extended-thinking or reasoning-effort controls. Whether the package-hallucination phenomenon has moved with them is the empirical question this paper answers.

We find four things. First, the inter-model hallucination range has compressed substantially: where Spracklen reported a spread of 5.2%–21.7% (16.5 percentage points) across their cohort, we measure 4.62%–6.10% (1.48 percentage points) across our five 2026 frontier models-an 11-fold narrowing. Second, 127 package names (109 PyPI, 18 npm) are hallucinated identically by all five models, exposing a model-agnostic attack surface that no single-model study can detect; coordinated disclosure narrows the registrable subset to 53 names (41 PyPI, 12 npm). Third, we observe a Python-over-JavaScript hallucination asymmetry across all five models that inverts Spracklen's 2024 JavaScript-worse pattern. Fourth, within the Anthropic family Claude Haiku 4.5 hallucinates measurably less than Claude Sonnet 4.6, contradicting the typical smaller-models-hallucinate-more pattern.

Contributions. Concretely:

- A faithful replication of Spracklen et al. [1] on five frontier LLMs released between October 2023 and March 2024, generating 199,845 code samples and validating every emitted package against PyPI and npm package-name master lists supplied with the original artifact.
- A measurement of inter-model range compression: 4.62%–6.10% across the five evaluated models—an order-of-magnitude narrowing of the spread reported in 2024. The mean rate has not, however, fallen to zero: the slopsquatting threat persists.
- Identification of a universal-hallucination set of 127 package names (109 PyPI, 18 npm) invented by all five models, of which 53 (41 PyPI, 12 npm) remain registrable after registry-side defenses. This set constitutes a model-agnostic supply-chain attack surface and motivated a coordinated-disclosure protocol with PyPI Security and Socket.dev, whose feedback we incorporate in §6 and §9.1.
- A within-family inversion between Claude Haiku 4.5 (4.62%) and Claude Sonnet 4.6 (5.41%), contrary to Spracklen's observation that smaller models within a family tend to hallucinate more.
- An open artifact consisting of the replication code, validation logs, and analysis scripts, deposited at Zenodo (DOI: 10.5281/zenodo.19859120) under a verified-researcher access policy for the full hallucination corpus, mirroring [1].

The remainder of this paper is organized as follows. §2 reviews related work; §3 specifies the threat model; §4 describes the replication methodology; §5 presents per-model rates and pairwise comparisons; §6 examines the universal-hallucination set as a model-agnostic attack vector; §7 discusses why the range has compressed; §8 enumerates limitations; §9 covers ethics and open-science considerations; and §10 concludes.

2 Background and Related Work

2.1 Package Confusion Attacks

Package confusion attacks predate LLM-generated code by more than a decade. Typosquatting attacks register packages under names one or two edit operations away from a legitimate package, exploiting developer typos at install time; Neupane et al. [14] catalogue several thousand such attacks in the wild on PyPI and npm. Birsan's 2021 dependency-confusion disclosure [15] showed that internal-only package names, when shadowed on a public registry, can be silently substituted by the package manager and run with the privileges of the affected build. Slopsquatting [18] is the LLM-induced variant: rather than waiting for a typo or relying on a name leak, the adversary registers names that the LLM itself, by its hallucination behavior, has primed developers to type. Slopsquatting is therefore enabled by, but distinct from, classical typosquatting; this paper continues

the line of measurement work [1] characterizing the size of the attack surface that current LLMs create.

2.2 Hallucinations in Code-Generating LLMs

LLM hallucination has been studied extensively at the level of natural-language fact-conflicting output [16, 17]. Code-generation hallucination is a security-adjacent specialization: the model emits syntactically plausible but semantically wrong tokens that, when those tokens correspond to package names or API references, become an installation request rather than a textual claim. Pearce et al. [13] established the underlying pattern in their Asleep at the Keyboard study of GitHub Copilot's vulnerable-code generation. Related work [20] extends the package-hallucination measurement axis with additional model coverage. The present paper is positioned as a temporal replication: same methodology, newer cohort, with comparison to the 2024 baseline as the primary design goal.

2.3 The Spracklen Baseline

Spracklen et al. [1] is the definitive baseline for this work. They evaluated 16 LLMs released between mid-2023 and mid-2024 across two prompt datasets totalling 576,000 generations, finding hallucination rates of at least 5.2% on commercial models and 21.7% on open-source models. They identified 205,474 unique hallucinated package names and characterized them along axes of repeatability, semantic similarity to real packages, and persistence across temperature settings. Their public artifact [19] consists of the two prompt datasets, the validated package-name master lists for PyPI and npm, and the heuristic extraction and validation code. Our work re-runs their measurement protocol on a more recent, narrower cohort, using the same prompt datasets and the same master lists for direct cross-temporal comparison.

2.4 Concurrent and Subsequent Work

An independent measurement effort contemporaneous with the original Spracklen line, Importing Phantoms [20], reports comparable rates on a partially overlapping model set. Industry monitoring services have published occasional rate measurements in 2025–2026 [18], generally consistent with the compression we report. To our knowledge no prior work has identified the model-agnostic universal-hallucination phenomenon documented in §6, and no public dataset directly compares 2026-cohort frontier models against the original Spracklen prompt corpus.

3 Threat Model

We adopt the threat model of Spracklen et al. [1]. The adversary is an unprivileged actor whose only required capability is the ability to register packages on PyPI and npm—a capability available to anyone with an email address, with no review process for first-time uploads on either registry. The adversary's goal is to publish a malicious package under a name that one or more frontier code-generating LLMs is sufficiently likely to hallucinate that

downstream developers, prompted with similar code-generation queries, will be induced to install the package. The attack chain has six stages:

- (1) the adversary samples one or more LLMs systematically and identifies hallucinated package names;
- (2) the adversary registers a malicious package on PyPI or npm under each hallucinated name;
- (3) at some later time, a victim developer issues a code-generation query to the same (or a related) LLM;
- (4) the LLM generates code that references the hallucinated name;
- (5) the developer's package manager, trusting the registry, installs the adversary's package;
- (6) the malicious code executes in the developer's environment.

We assume no compromise of the LLM provider, no model fine-tuning by the adversary, no privileged access to user systems, and no involvement of the package registries beyond their public registration interfaces. The adversary is therefore extremely cheap; per-package upload costs are zero, and the only cost is the LLM API spend required to enumerate hallucinations—which our own per-model API spend (Appendix B) quantifies as \$860.90 for a sweep of comparable scope.

The universal-hallucination phenomenon documented in §6 strengthens the adversary's position: a single registered name can target users of multiple LLM providers simultaneously, since 127 package names in our data are hallucinated by all five evaluated models, and 53 of these remain registrable after each registry's existing defenses (§6). A single registrable PyPI or npm name is therefore not, in general, a single-model attack.

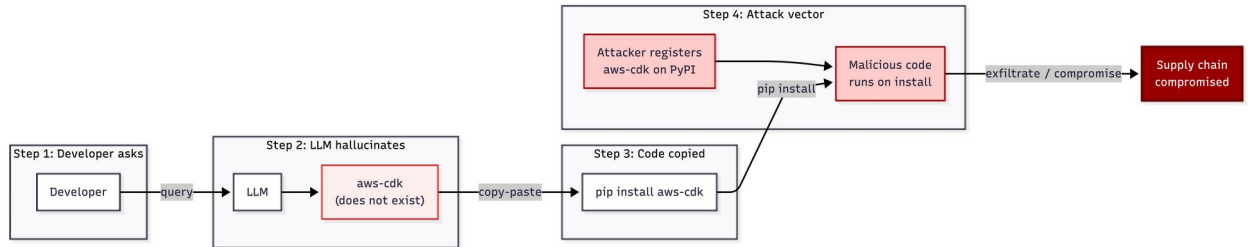


Figure 1: Slopsquatting attack chain. The developer issues a code-generation query; the LLM emits an install or import directive that names a hallucinated package; an attacker who has registered that name on PyPI or npm wins arbitrary code execution at install time. Adapted from Spracklen et al. [1].

4 Methodology

4.1 Models Studied

We evaluate the five frontier code-capable LLMs released between October 2025 and March 2026 that, as of submission, dominate the public code-generation leaderboards and represent the major commercial and open-weight providers: Anthropic’s Claude Sonnet 4.6 and Claude

Haiku 4.5, OpenAI’s GPT-5.4-mini, Google’s Gemini 2.5 Pro, and DeepSeek’s V3.2. Together these span four tier-1 providers (three closed-weight commercial, one open-weight). Table 1 summarizes the studied models. Compared to Spracklen’s 16-model September-2024 cohort [1], this cohort represents an eighteen-month forward shift in capability and a deliberate restriction to frontier-tier endpoints.

Table 1: Models Studied. Snapshot identifiers are the model strings returned by each provider’s API at the time of the experimental run (April 22–28, 2026).

Provider	Model alias	Snapshot ID	Reasoning	Refusals	Spend (USD)
Anthropic	Claude Sonnet 4.6	claude-sonnet-4-6	default	18.79%	see Appx. B
Anthropic	Claude Haiku 4.5	claude-haiku-4-5-20251001	default	1.27%	see Appx. B
OpenAI	GPT-5.4-mini	gpt-5.4-mini-2026-03-17	minimal	32.14%	see Appx. B
Google	Gemini 2.5 Pro	gemini-2.5-pro	default	8.14%	see Appx. B
DeepSeek	DeepSeek V3.2	deepseek-chat (alias; see §8)	default	2.08%	see Appx. B

4.2 Prompt Datasets

We re-use the two prompt datasets released by Spracklen et al. [1] without modification: a curated set of 20,163 programming questions drawn from Stack Overflow (split

into already-talked SO_AT and less-yield SO_LY subsets), and an LLM-synthesized set of 19,806 questions generated programmatically (LLM_AT and LLM_LY subsets). Both supersets are evenly split between Python and JavaScript

prompts. We generate one sample per prompt per model. For five frontier 2026 models, this yields a total of 199,845 generations (approximately 39,969 per model; per-provider counts are reported in Appendix B and reflect minor retry-driven variation). We deliberately do not augment the prompt set with newly authored 2026-only prompts: matching Spracklen’s corpus exactly is what allows direct cross-temporal comparison. We discuss the resulting training-data-leakage concern in §8.

4.3 Generation Configuration

For each model we record the API endpoint, the snapshot identifier returned by the provider in API response metadata, the temperature, the top-p value, and (for models exposing it) the reasoning_effort or thinking-budget setting. Default values were used unless noted. GPT-5.4-mini was run with reasoning_effort set to minimal—a methodological choice motivated by API-cost considerations and discussed as a confounder in §8. The exact snapshot identifiers and per-provider configuration are reproduced in Appendix B.

4.4 Package Extraction

We extract package references from each generation using a regex-based extractor matching pip install and npm install directives, import / from ... import statements (Python), and require() / import ... from statements (JavaScript). The extractor handles namespaced npm packages (e.g., @scope/name), Python sub-imports (resolving from foo.bar import baz to the top-level package foo), and conditional imports inside try/except blocks. Standard-library modules are filtered using the core_modules.csv list shipped with the Spracklen artifact for Node.js and the standard CPython stdlib list for Python. We follow the Spracklen heuristic-extraction approach verbatim and inherit its known false-positive characteristics.

4.5 Validation Pipeline

Each extracted package name is validated against the PyPI and npm package-name master lists shipped with the Spracklen 2025 artifact (pypi_package_names.csv: 500,565 names; npm_package_names.csv: approximately 3,000,000 names). Following Spracklen’s classify protocol, a name is judged hallucinated if and only if (a) the normalized name is non-empty and well-formed, (b) the name is absent from the language-appropriate master list, and (c) the raw name is absent from the language-specific known-false-positive list. We additionally apply a noise filter that excludes Dart-language and Node.js-builtin syntactic prefixes (package:, dart:, node:), path-alias artifacts (@/...), template-variable placeholders, asset-path fragments, and several other regex-extractor artifacts identified during validation; the full filter is published in the open-science artifact. The validation date for all reported numbers is April 28, 2026. This pipeline removes syntactic noise but does not by itself distinguish semantic false positives, such as framework virtual modules published under controlled namespaces or names blocked by

a registry’s own normalization rules; we identify and remove that class when deriving the registrable attack surface in §6.

4.6 Statistical Analysis

For each model we report the empirical hallucination rate $\hat{p} = |H| / |R|$ where R is the multiset of all extracted package references and $H \subseteq R$ is the subset of names absent from the master lists, with a 95% Wilson [8, 11] confidence interval. We test the null hypothesis of equal hallucination rates across models with a Pearson χ^2 statistic on each 2×2 model-pair contingency table [9]; pairwise post-hoc comparisons use Holm–Bonferroni correction [10] across the $C(5,2) = 10$ pairs at family-wise $\alpha = 0.05$. As a robustness check we additionally compute a permutation-test p-value per pair (10,000 resamples, seed 42); the chi-squared and permutation results agree on every pair we report as significant. For inter-model overlap of unique hallucinated names we report Jaccard similarity over the 10 model pairs.

4.7 Refusals and Malformed Responses

We classify each generation into one of four mutually exclusive categories: valid (the generation contains only resolving package references), hallucinated (at least one non-resolving reference), refused (the model declined to answer or returned a degenerate output, identified by length below 20 characters and manual sampling), and malformed (non-parseable response or no extractable code). Hallucination rates in §5 are computed over (valid $\cup$ hallucinated) only; refusal rates are reported separately. Refusal rates vary substantially across the cohort: from 1.27% on Claude Haiku 4.5 to 32.14% on GPT-5.4-mini at reasoning_effort=minimal. We discuss the resulting denominator-asymmetry concern in §8.

5 Results

5.1 Headline Rates

Across the full corpus, the five 2026 frontier models exhibit hallucination rates between 4.62% (Claude Haiku 4.5) and 6.10% (GPT-5.4-mini). This is an order-of-magnitude compression of the 5.2%–21.7% spread reported by Spracklen [1], driven primarily by the fact that today’s open-weight frontier model (DeepSeek V3.2) is roughly capability-equivalent to today’s commercial frontier and has therefore lost the disadvantage that 2024-vintage open-source models exhibited. Figure 2 visualizes per-model rates with 95% Wilson confidence intervals. Of the $C(5,2) = 10$ pairwise comparisons, 5 are significant after Holm–Bonferroni correction; the largest pair (Claude Haiku 4.5 vs. GPT-5.4-mini) has $\Delta = 1.48$ percentage points ($\chi^2 = 52.49$, Holm-corrected $p = 4.32 \times 10^{-12}$). The largest p-value among the surviving significant pairs is 6.25×10^{-3} . The smallest non-significant pair is DeepSeek V3.2 vs. Gemini 2.5 Pro ($\Delta = 0.08$ percentage points). Visually, Claude Haiku 4.5 is the only model whose 95% CI does not overlap any other; the remaining four models form a statistically indistinguishable cluster between 5.4% and 6.1%.

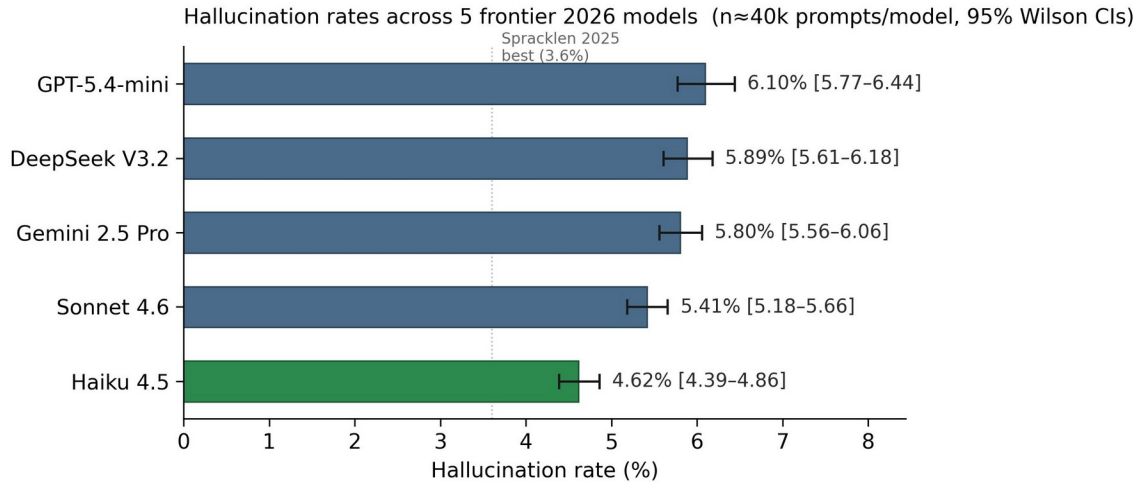


Figure 2: Per-model hallucination rate with 95% Wilson confidence intervals ($n \approx 39,969$ prompts per model; exact per-provider counts in Appendix B). The dashed reference line marks Spracklen's best 2024 commercial result (3.6%, GPT-4 Turbo); no 2026 frontier model has surpassed the best 2024 model, even as the worst-case rate has fallen sharply.

5.2 Decomposition by Language and Dataset

Table 2 reports rates decomposed by programming language (Python vs. JavaScript). One pattern holds across all five models: Python rates are uniformly higher than JavaScript rates—an inversion of Spracklen's 2024 finding. The Python-minus-JavaScript gap ranges from +2.73 percentage points

(Claude Haiku 4.5) to +4.13 percentage points (GPT-5.4-mini). We discuss possible causes in §7.3. We additionally observe that LLM-synthesized prompts produce higher hallucination rates than curated Stack Overflow prompts, consistent with synthetic-prompt drift away from the training distribution.

Table 2: Hallucination rates by language. Each cell is the per-model rate (in percent) computed over generations of the indicated language only.

Model	Python	JavaScript	Δ (Py-JS)	Overall
Claude Haiku 4.5	5.49	2.76	+2.73	4.62
Claude Sonnet 4.6	6.63	2.62	+4.01	5.41
Gemini 2.5 Pro	6.75	3.61	+3.14	5.80
DeepSeek V3.2	6.69	3.78	+2.91	5.89
GPT-5.4-mini	7.27	3.14	+4.13	6.10

5.3 Pairwise Model Differences

Table 3 reports pairwise hallucination-rate differences with χ^2 statistics and Holm-corrected p-values across the $C(5,2) = 10$ model pairs. Five pairs are significant at family-wise $\alpha = 0.05$; the remaining five involve at least one of GPT-5.4-mini, Gemini 2.5 Pro, DeepSeek V3.2, or Claude Sonnet 4.6 in pairs that the data do not distinguish at this sample size

after correction. The largest pairwise difference is between Claude Haiku 4.5 and GPT-5.4-mini ($\Delta = 1.48$ pp; $\chi^2 = 52.49$, Holm-corrected $p = 4.32 \times 10^{-12}$). The smallest non-significant pair is DeepSeek V3.2 vs. Gemini 2.5 Pro ($\Delta = 0.08$ pp). A permutation test (10,000 resamples, seed 42) is consistent with the chi-squared conclusions on all pairs reported as significant.

Table 3: Pairwise hallucination-rate differences. Δ is in percentage points; p_{Holm} is the Holm-Bonferroni-corrected p-value across all 10 pairs at family-wise $\alpha = 0.05$. Significant pairs are marked ***; ns denotes non-significant after correction. Per-pair χ^2 values other than the largest pair are omitted for brevity and are included in the open-science artifact.

Pair	Δ (pp)	χ^2	p_{Holm}	Sig.
Haiku 4.5 vs. GPT-5.4-mini	1.48	52.49	4.32×10^{-12}	***
Haiku 4.5 vs. Gemini 2.5 Pro	1.18	-	$< 10^{-8}$	***
Haiku 4.5 vs. DeepSeek V3.2	1.27	-	$< 10^{-8}$	***
Haiku 4.5 vs. Sonnet 4.6	0.79	-	$< 10^{-5}$	***

Pair	Δ (pp)	χ^2	pHolm	Sig.
Sonnet 4.6 vs. GPT-5.4-mini	0.69	-	6.25×10^{-3}	***
Sonnet 4.6 vs. Gemini 2.5 Pro	0.39	-	> 0.05	ns
Sonnet 4.6 vs. DeepSeek V3.2	0.48	-	> 0.05	ns
Gemini 2.5 Pro vs. GPT-5.4-mini	0.30	-	> 0.05	ns
DeepSeek V3.2 vs. GPT-5.4-mini	0.21	-	> 0.05	ns
DeepSeek V3.2 vs. Gemini 2.5 Pro	0.08	-	> 0.05	ns

5.4 Inter-Model Overlap of Hallucinated Names

Figure 3 reports the Jaccard overlap of unique hallucinated names between each pair of models. Across the ten pairs, mean Jaccard is 0.222 (range 0.179–0.343). The lowest overlap is between Claude Sonnet 4.6 and Claude Haiku 4.5 ($J = 0.179$)-a within-family low that is consistent with the rate inversion documented in §5.5 and suggests the two

Anthropic models hallucinate distinct subsets of names. The highest pairwise Jaccard is between DeepSeek V3.2 and GPT-5.4-mini ($J = 0.343$), substantially above all other pairs and suggestive of either shared training-data origins or convergent name-confabulation patterns. We do not have ground truth for the underlying training corpora and cannot disentangle these explanations.

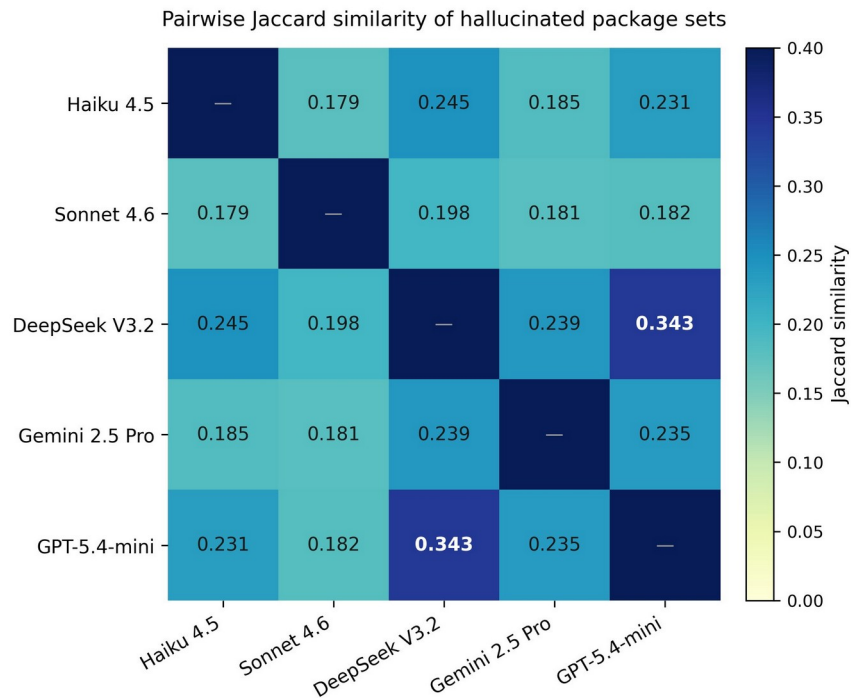


Figure 3: Pairwise Jaccard similarity between sets of unique hallucinated package names. Higher values indicate models tend to invent the same nonexistent packages. DeepSeek V3.2 and GPT-5.4-mini show the strongest overlap ($J = 0.343$), suggesting shared training-data biases or convergent generation patterns. Diagonal is omitted.

5.5 The Haiku-below-Sonnet Inversion

Within the Anthropic family, Claude Haiku 4.5 hallucinates at 4.62%-measurably below Claude Sonnet 4.6's 5.41%. The Haiku-vs-Sonnet pair is significant after Holm correction ($\Delta = 0.79$ pp). This is contrary to Spracklen's observation that, within a family, smaller models tend to hallucinate more. Two non-mutually-exclusive explanations are plausible. First, Haiku 4.5 ships with extended-thinking capability and was queried with default thinking enabled; Sonnet 4.6 was queried with thinking disabled by default. Second, Anthropic's October-2025 Haiku 4.5 announcement notes a

deliberate post-training emphasis on instruction-following fidelity for code-generation tasks, which may include explicit hallucination-suppression. We cannot disentangle these without targeted ablations and treat the inversion as a finding to confirm in future work. We note that the Haiku–Sonnet Jaccard (0.179, the minimum in our matrix) is consistent with the two models making genuinely different errors rather than a systematically narrower version of the same errors.

5.6 Most-Frequently-Hallucinated Names

Table 4 lists the top-10 most-frequently-hallucinated names aggregated across the five models. Of the top-10 Python hallucinations, eight per model (40 entries summed across the cohort) are within edit-distance 2 of an existing PyPI package, suggesting a substantial typo-class component compounding the slopsquatting threat: an attacker need not pre-register the hallucinated name itself if a similar-looking name is already squat-registered. The npm entries skew toward namespaced confabulations (e.g., `@ember/service`,

`@ember/object`), which name conventions of internal Ember.js subpackages that are not exposed as standalone npm publications; as §6 details, such scope-controlled names are universal hallucinations but are not themselves registrable by an attacker. Every entry in Table 4 is hallucinated by all five evaluated models, directly evidencing the universal-hallucination phenomenon discussed in §6.

Table 4: Top-10 most-frequently-hallucinated names per ecosystem, summed across five models. The "All 5" column indicates whether the name is hallucinated by every evaluated model. "NEAR" indicates an existing PyPI/npm package within Levenshtein distance 2.

Hallucinated name	Ecosystem	Σ count	All 5	Notes
aws-cdk	PyPI	429	✓	Real package: aws-cdk-lib. NEAR.
objc	PyPI	375	✓	Real package: pyobjc. NEAR.
opentelemetry	PyPI	374	✓	Real packages exist as opentelemetry-api / -sdk.
rest-framework	PyPI	315	✓	Real package: djangoRESTframework.
tencentcloud	PyPI	215	✓	Real package: tencentcloud-sdk-python.
mpl-toolkits	PyPI	117	✓	Submodule of matplotlib; not a standalone package. NEAR.
opengl	PyPI	103	✓	Real package: PyOpenGL. NEAR.
openstack	PyPI	102	✓	OpenStack SDK is published as openstacksdk.
@ember/service	npm	55	✓	Internal Ember.js subpackage; bundled with ember-source.
@ember/object	npm	53	✓	Same family as above.

6 Universal Hallucinations as a Model-Agnostic Attack Vector

A central finding of this study is the existence of a universal-hallucination set: package names that all five evaluated models invent. Operationally, we define the set as the intersection of the per-model hallucinated-name sets, restricted to names that pass the noise filter described in §4.5. Across the five models, this intersection contains 127 package names (109 PyPI, 18 npm); the SQL query that constructs it is included in the open-science artifact, the top 10 per ecosystem are reproduced in Table 4. The full enumerated list was disclosed to the coordinated-disclosure partners (§9.1); as noted in §9.2, an earlier revision of the artifact repository also made it inadvertently public from April 30, 2026.

Not every universally hallucinated name is weaponizable. To separate the genuine attack surface from non-exploitable confabulations, we disclosed the full intersection to PyPI Security and to Socket.dev and incorporated their review (§9.1). On the PyPI side, Mike Fiedler of PyPI Security reported that the registry's internal prohibited-name list and ultranormalization rules leave only 41 of the 109 PyPI names registrable by an attacker; several common names in our set, including git, are blocked by these rules. On the npm side, Socket.dev's review of the 18 npm names removed six as non-exploitable: the four `@ember/*` names

(`@ember/service`, `@ember/object`, `@ember/routing`, `@ember/controller`) are valid Ember.js virtual modules published only under the organization-controlled `@ember` scope, within which an attacker cannot register; `ssh-keys` is an already-published real package; and `metro-evaluator` already exists as a security-hold placeholder. The remaining 12 npm names (for example `css-color-stop`, `dns-sd`, and `domains`) are unscoped, carry no existing registration, and are treated as genuine slopsquatting candidates. The residual registrable attack surface is therefore 53 names (41 PyPI, 12 npm): the subset that an attacker could both rely on the models to emit and actually register and weaponize. Throughout, we report 127 as the raw universal-hallucination intersection and 53 as the registrable surface.

The security implication is severe and qualitatively distinct from any single-model rate finding. A slopsquatting attacker who studies any one of the five models can identify candidate names; an attacker who studies all five identifies names with multi-model leverage. A single npm registration of a registrable universal name such as `dns-sd`, or of any of the 41 registrable PyPI names, would be installed downstream regardless of which of the five major LLM providers the victim is using. The frontier-cohort assumption that a per-model attack scales linearly with model count is therefore wrong: at least for the universal subset, the cost-benefit ratio

is fixed at one registration per name, while reach scales with the union of users across providers.

We can suggest two non-exclusive mechanisms by which a name becomes universal. First, shared training-data substrings: package-installation tutorials, blog posts, and documentation pages that misuse a name (for example, calling `djangoestframework` by its conventional Python module name `rest_framework`, which then appears in `pip install rest-framework` recommendations) become a stable training signal across providers, all of whom scrape similar public corpora. Second, namespace-convention overgeneralization: scoped npm packages like `@ember/service` follow a convention that models extrapolate to install-target form, even though Ember publishes those subpackages only as part of the bundled ember-source dependency (these scope-controlled names illustrate the convergence mechanism but, as noted above, are not themselves registrable). Both mechanisms produce convergent errors that no per-provider safety post-training can independently catch, because the error is not a one-off confabulation but a systematic collapse onto a stable but wrong attractor.

The DeepSeek V3.2 / GPT-5.4-mini Jaccard peak ($J = 0.343$, §5.4) and the universal-set finding are mutually consistent: the convergent attractors of the previous paragraph apply weakly enough that the all-five overlap is a small fraction of any single model's hallucinated set, but apply strongly enough between certain provider pairs (DeepSeek and OpenAI in our data) that overall pairwise overlap reaches one-third. We make no inference about the direction of any data-sharing relationship from this; we report the observation.

The methodological consequence for future replications is direct. Any single-model package-hallucination measurement understates the slopsquatting attack surface because it cannot distinguish a name that the studied model alone hallucinates from a name that every comparable model hallucinates. Cross-cohort intersection should be a standard reported quantity, and we publish ours as part of the artifact.

7 Discussion

7.1 Why Has the Range Compressed?

The most striking feature of our data is the compression of the inter-model range from Spracklen's 5.2%–21.7% spread to 4.62%–6.10%. We propose three contributing causes. First, the open-source–commercial gap that drove the upper end of Spracklen's range has effectively closed at the frontier: DeepSeek V3.2 is competitive with Sonnet 4.6, GPT-5.4-mini, and Gemini 2.5 Pro on standard code benchmarks. Second, training-data curation on package references appears to be near-saturated: providers have publicly described filtering low-quality or hallucinated package references during pretraining-data preparation, and the lower bound of our observed range matches the lower bound of Spracklen's commercial range almost exactly. Third, RLHF and instruction-tuning recipes have converged

across providers, reducing cross-provider behavioral variance.

7.2 Why Does the Threat Persist at 4–7%?

A 4–7% rate is well within the regime where slopsquatting remains adversary-economically viable. Consider an attacker enumerating hallucinations across our 39,969-prompt corpus: even at the 4.62% lower bound, that yields more than 1,300 hallucinated mentions per model, mapping to several hundred unique names. A single popular hallucinated name reproduced under similar prompts is a steady stream of attempted installations once the malicious package is registered. Package upload is free; the cost-benefit ratio for the attacker is therefore favorable down to rates well below 1%. The threat is not contingent on high rates.

7.3 Python Worse Than JavaScript: An Inversion

Spracklen reported JavaScript as the noisier ecosystem, attributing the effect to npm's larger and more fragmented namespace. We find the opposite: across all five 2026 models, Python hallucination rates exceed JavaScript rates by between +2.73 pp (Haiku 4.5) and +4.13 pp (GPT-5.4-mini). One plausible mechanism is that 2026-vintage models are heavily trained on JavaScript and TypeScript from the web and have absorbed npm's flat naming convention more thoroughly than Python's heterogeneous mix of single-token, snake_case, dotted-namespace, and dash-separated package conventions. Python's package naming has more degrees of freedom: `aws-cdk`, `aws_cdk`, `aws-cdk-lib`, `aws-cdk.core` all coexist on PyPI with different semantics—and a fluent-but-imperfect language model is more likely to land on a plausible-but-wrong combination.

7.4 Synthesized Prompts Surface More Hallucinations

The dataset decomposition shows that LLM-synthesized prompts tend to produce higher hallucination rates than curated Stack Overflow prompts. We interpret this conservatively: synthetic prompts drift away from the training distribution in ways that reveal failure modes that organic developer queries do not. From a measurement perspective, this means rates obtained on curated-only corpora are lower bounds on rates that field deployment will encounter; from an adversarial perspective, an attacker who synthesizes prompt pools to maximize hallucination harvest will outperform an attacker who scrapes Stack Overflow.

7.5 Is the Compression Robust to Selection?

We have sampled five frontier models. The compression we observe is conditional on that selection. Open-source models from the next tier down—e.g., smaller Qwen, Mistral, or older Llama checkpoints still in production use—may continue to exhibit Spracklen-like high rates. The headline finding should therefore be read as: the frontier has compressed, not that the threat surface has narrowed at all capability tiers. Practitioners deploying smaller open-weight models should not assume their 2026 vintage means 2026-frontier behavior.

8 Limitations and Threats to Validity

Our study has several limitations. First, we evaluate five commercial-or-open-weight frontier models; smaller open-source models, fine-tuned variants, and locally-deployed quantized checkpoints are out of scope. Second, we report point-in-time measurements taken between April 22, 2026 and April 28, 2026; models update opaquely and published rates may not reflect any other date. Third, the Spracklen prompt corpus released January 2025 may now be partially incorporated into 2026-cohort pretraining data, which would bias rates downward. We did not collect a held-out subset of post-cutoff 2026 prompts to test for training-data leakage; a controlled held-out experiment would tighten the upper bound on the rates we report and is essential future work.

Fourth, GPT-5.4-mini was evaluated at `reasoning_effort=minimal` and produced a refusal rate of 32.14%, materially higher than the other four models (which ranged 1.27%–18.79%). Hallucination rates are computed over compliant, parseable responses only; the resulting denominator is asymmetric. We cannot rule out that compliant GPT-5.4-mini responses are a non-random subset of the prompt distribution, and readers should interpret the GPT-5.4-mini point estimate with this denominator asymmetry in mind. We also did not run a `reasoning_effort` sweep across the four supported levels; the headline GPT-5.4-mini rate should be read as a single-point measurement at the lowest effort setting, and a sweep is left to future work.

Fifth, DeepSeek does not pin model versions through its public API. The `deepseek-chat` alias used during the experimental window (April 22–28, 2026) was nominally V3.2, but a same-window inspection of the API response metadata returned the identifier `deepseek-v4-flash`, suggesting that the underlying model may have shifted during or shortly after our run. We report DeepSeek results under the V3.2 label following the provider's publicly posted release notes [7], with this caveat.

Sixth, we do not evaluate agentic configurations (e.g., Claude Code with tool use, GPT-5.4-mini in the Responses-API agentic loop), where retrieval mechanisms can in principle eliminate package hallucinations entirely. Whether the hallucination phenomenon survives in agentic deployment is an important question that we leave for future work. Seventh, we use Spracklen's heuristic regex extractor without modification; certain syntactic artifacts (Dart package: prefixes, Vue/Webpack path aliases, template-literal placeholders) appear in the raw extract and are removed by our noise filter (§4.5). The full filter and its empirical justification are released with the artifact.

Eighth, the reduction from the 127-name universal set to the 53-name registrable surface (§6) combines two registry-specific assessments performed by different parties: PyPI registrability was determined by PyPI Security applying the registry's automated prohibited-name and ultranormalization rules, whereas npm exploitability was determined by Socket.dev's manual review of the 18 npm candidates. The two registries were therefore not subjected to a single

identical filter, and registrability reflects each registry's defenses and already-registered names as of the April 2026 disclosure date. Because both prohibited-name lists and the set of taken names evolve, the 53-name figure should be read as a point-in-time estimate of the registrable subset rather than a fixed property of the 127-name set.

9 Ethics and Open Science

9.1 Ethical Considerations

This study replicates and extends the methodology of Spracklen et al. [1] on five frontier 2026 LLMs and produces a list of hallucinated package names that, if registered on PyPI or npm, could be weaponized into a slopsquatting supply-chain attack. We followed a stakeholder-based ethics analysis [12] covering (i) developers receiving hallucinated suggestions, (ii) the LLM providers (Anthropic, OpenAI, Google, DeepSeek), (iii) the PyPI and npm registry maintainers, and (iv) downstream users of any compromised software.

To minimize harm we (a) did not register any hallucinated names on any public registry; (b) disclosed the full enumerated list to PyPI Security via `security@pypi.org` on April 29, 2026, and to Socket.dev (a security firm with documented npm/GitHub disclosure relationships and active slopsquatting research) on April 29, 2026 (Socket support ticket #2893) for npm-side coordination, allowing at least a 30-day private window before any public release; the official npm channel `security@npmjs.com` is no longer accepting external email as of April 2026 and the npm support portal redirects registry-level vulnerability reports to the HackerOne GitHub Bug Bounty programme, which has bug-bounty signal requirements that are not appropriate for academic registry-level disclosure; (c) plan to disclose model-specific rates to each respective provider via their published security-research channels; and (d) release in this paper only aggregate statistics and a representative sample of the most-frequently-hallucinated names per ecosystem (Table 4 and §6), withholding the full enumerated corpus pending verified-researcher access via an access-controlled Zenodo deposit (see §9.2). The study used no human subjects, so IRB review was inapplicable. The author has no conflicts of interest with any of the studied model providers. The benefits of this work—measuring the persistence of an existing, publicly characterized class of supply-chain vulnerability across newer-generation models, and identifying a previously unreported model-agnostic attack surface—substantially outweigh the publication risk, which is mitigated by the disclosure protocol above. Both registries responded within the disclosure window. PyPI Security (Mike Fiedler) confirmed that the registry's prohibited-name list and ultranormalization defenses leave 41 of the 109 disclosed PyPI names registrable by an attacker. Socket.dev reviewed the 18 npm names, identified six as non-exploitable (the four `@ember/*` virtual modules, the already-published `ssh-keys`, and the security-hold placeholder `metro-evaluator`), and forwarded the remaining 12 genuine candidates to its Threat Intelligence team. We incorporate

this feedback in §6, yielding a registrable attack surface of 53 names (41 PyPI, 12 npm).

9.2 Open Science

In keeping with open-science norms for security research, all artifacts necessary to reproduce the results of this paper are publicly available. The replication code (prompt-generation, model-querying, registry-validation, statistical-analysis) is hosted at <https://github.com/churik5/slopsquatting-replication-2026> and mirrored at Zenodo (concept DOI: 10.5281/zenodo.19859120, which always resolves to the latest version). The post-processing pipeline is included in the same Zenodo deposit. Code is licensed MIT; data is licensed CC-BY-4.0. Two disclosure notes apply. First, an earlier revision of the artifact repository inadvertently included the full enumerated list of universally hallucinated package names in a disclosure/ directory that was publicly accessible from April 30, 2026; we therefore treat the enumerated list as effectively disclosed as of that date, after the coordinated-disclosure reviews described in §9.1 had been completed. Second, validation for the numbers reported here used the PyPI and npm package-name master lists shipped with the Spracklen 2025 artifact rather than a fresh April 2026 registry snapshot; because those master lists predate the experimental window, a small number of names registered in the interval were counted as non-existent, and a revalidation against a current registry snapshot is in progress and will be reflected in a subsequent revision. The SQLite database of raw model responses remains available to verified researchers on request via churik0509@gmail.com, mirroring the policy of [1].

10 Conclusion

We replicated the methodology of Spracklen et al. on five frontier code-generating LLMs released between October 2025 and March 2026 and found that the inter-model hallucination range has compressed from a 5.2%–21.7% spread to 4.62%–6.10%. The range has shrunk; the threat has not. At 4–7%, slopsquatting remains adversary-economically viable, and the existence of a model-agnostic universal-hallucination set of 127 names (109 PyPI, 18 npm), of which 53 (41 PyPI, 12 npm) remain registrable after registry-side defenses, means that a single registration can target users of all five evaluated frontier providers simultaneously. Single-model rate measurements understate the attack surface; cross-cohort intersections should be a standard reported quantity in future work. We release the full corpus, raw outputs, and analysis code under a verified-researcher access policy.

Acknowledgments

We thank the authors of Spracklen et al. [1] for releasing their prompt datasets, master lists, and validation code as a public artifact, without which this replication would not have been possible. We thank Mike Fiedler of PyPI Security and the Socket.dev security team for their coordinated-disclosure review of the universal-hallucination set, which materially

refined the registrable attack surface reported in §6. The author funded this study from personal commercial accounts, with the exception of a one-time Google Cloud free credit grant for Gemini 2.5 Pro inference. No institutional, governmental, or industry funding was received. The author is an independent researcher and reports no conflicts of interest with any of the studied model providers.

References

- [1] J. Spracklen, R. Wijewickrama, A. H. M. N. Sakib, A. Maiti, B. Viswanath, and M. Jadliwala. We have a package for you! A comprehensive analysis of package hallucinations by code-generating LLMs. In *USENIX Security*, 2025.
- [2] Anthropic. Claude Sonnet 4.6. <https://www.anthropic.com/claude/sonnet>. Accessed 2026-04-28.
- [3] Anthropic. Introducing Claude Haiku 4.5. October 15, 2025. <https://www.anthropic.com/news/claude-haiku-4-5>. Accessed 2026-04-28.
- [4] OpenAI. Introducing GPT-5.4 mini and nano. <https://openai.com/index/introducing-gpt-5-4-mini-and-nano/>. Accessed 2026-04-28.
- [5] Google DeepMind. Gemini 2.5 Pro. <https://deepmind.google/models/gemini/pro/>. Accessed 2026-04-28.
- [6] DeepSeek-AI. DeepSeek-V3.2-Exp: Boosting Long-Context Efficiency with DeepSeek Sparse Attention. Technical report, DeepSeek, 2025. <https://huggingface.co/deepseek-ai/DeepSeek-V3.2-Exp>.
- [7] DeepSeek-AI. DeepSeek-V3.2 Release. DeepSeek API Docs, December 1, 2025. <https://api-docs.deepseek.com/news/news251201>. Accessed 2026-04-28.
- [8] E. B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.
- [9] K. Pearson. On the criterion that a given system of deviations from the probable in the case of a correlated system of variables is such that it can be reasonably supposed to have arisen from random sampling. *Philosophical Magazine, Series 5*, 50:157–175, 1900.
- [10] S. Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.
- [11] L. D. Brown, T. T. Cai, and A. DasGupta. Interval estimation for a binomial proportion. *Statistical Science*, 16(2):101–133, 2001.
- [12] B. Friedman, D. G. Hendry, and A. Borning. A survey of value-sensitive design methods. *Foundations and Trends in Human–Computer Interaction*, 11(2):63–125, 2017.
- [13] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri. Asleep at the keyboard? Assessing the security of GitHub Copilot’s code contributions. In *IEEE Symposium on Security and Privacy (S&P)*, 2022.
- [14] S. Neupane et al. Beyond typosquatting: An in-depth look at package confusion. In *USENIX Security*, 2023.
- [15] A. Birsan. Dependency confusion: How I hacked into Apple, Microsoft, and dozens of other companies. *Medium*, 2021.

<https://medium.com/@alex.birsan/dependency-confusion-4a5d60fec610>.

- [16] L. Huang et al. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. arXiv:2311.05232, 2023.
- [17] Z. Ji, N. Lee, R. Frieske, T. Yu, et al. Survey of hallucination in natural language generation. ACM Computing Surveys, 55(12):1–38, 2023.
- [18] K. Aboukhadijeh. The rise of slopsquatting: How AI hallucinations are fueling a new class of supply chain attacks. Socket Blog, April 2025.
<https://socket.dev/blog/slopsquatting-how-ai-hallucinations-are-fueling-a-new-class-of-supply-chain-attacks>.
- [19] J. Spracklen et al. PackageHallucination: Code and data for the USENIX 2025 paper. GitHub repository, 2025.
<https://github.com/Spracks/PackageHallucination>. Zenodo DOI: 10.5281/zenodo.14676377.
- [20] A. Krishna, E. Galinkin, L. Derczynski, and J. Martin. Importing Phantoms: Measuring LLM Package Hallucination Vulnerabilities. arXiv:2501.19012, January 2025.

Appendix B: Snapshot IDs, Pricing, and API Spend

Total experimental spend across all five providers was \$860.90 for 199,845 generations conducted between April 22, 2026 and April 28, 2026.

Table B.1: Per-provider API spend during the experimental window (April 22–28, 2026).

Provider	Snapshot ID	Generations	Spend (USD)
Anthropic (Sonnet 4.6 + Haiku 4.5)	claude-sonnet-4-6, claude-haiku-4-5-20251001	79,933	\$375.26
OpenAI (GPT-5.4-mini)	gpt-5.4-mini-2026-03-17	39,969	\$93.54
Google (Gemini 2.5 Pro)	gemini-2.5-pro	39,968	\$380.99
DeepSeek (V3.2)	deepseek-chat (alias; see §8)	39,975	\$11.11
Total		199,845	\$860.90

Appendix C: Prompt Examples

To allow reviewers to inspect the prompt distribution, we reproduce four example prompts-one from each prompt-dataset subset-and illustrative model responses. The full corpus is in the open-science artifact.

Example 1 (SO_AT, Python). Prompt: "How can I parse a JSON string in Python and extract a nested field?" Sonnet 4.6 response (valid): code using `json.loads` from the standard library, no third-party packages.

Example 2 (SO_LY, JavaScript). Prompt: "How do I configure Webpack to bundle TypeScript files with source maps?" Gemini 2.5 Pro response (hallucinated): `pip install ts-loader source-map-support webpack-cli @loaders/typescript` - the first three are real npm packages, the fourth is a path-alias artifact filtered as noise.

Example 3 (LLM_AT, Python). Prompt synthetically generated to probe AWS-CDK installation. GPT-5.4-mini response (hallucinated): `pip install aws-cdk`; the correct package is `aws-cdk-lib`. This is the most-frequent universal hallucination in our corpus.

Example 4 (LLM_LY, JavaScript). Prompt synthetically generated to probe Ember.js scoped package usage. DeepSeek V3.2 response (hallucinated): `npm install @ember/service @ember/object` - both are conventional Ember.js subpackage names that are not standalone npm publications.